Artificial Analysis Benchmarking Methodology

Scope

Artificial Analysis performs intelligence, quality, performance, and price benchmarking on AI models, inference API endpoints, and systems. This section of our website describes our benchmarking methodology, including both our quality benchmarking and performance benchmarking.

For our language model benchmarking, we note that we consider endpoints to be serverless when customers only pay for their usage, not a fixed rate for access to a system. Typically this means that endpoints are priced on a per token basis, often with different prices for input and output tokens.

Across all modalities, our performance benchmarking measures the end-to-end performance experienced by customers of AI inference services and systems. This means that benchmark results are not intended to represent the maximum possible performance on any particular hardware platform, they are intended to represent the real-world performance customers experience across providers.

We benchmark both proprietary and open weights models.

Language Model Intelligence

Language Model Intelligence

Capability Indexes

Cyber Index

Decision Index

Small Language Model Intelligence

Endpoint Accuracy Index

Coding Agents

AA-3D

Progress Index v1

Speech, Image, Video & Music

Speech to Text

Text to Speech

Speech to Speech

Text to Image

Video Generation

Music Generation

Inference

Language Model API Performance

System Load Test (AA-SLT)

AA-AgentPerf

Misc.

Openness Index

Search API

AA-PersonalityBench

Definitions

On this page, and across the Artificial Analysis website, we use the following terms:

- **Model:** A large language model (LLM), including proprietary, open source, and open weights models.
- **Model Creator:** The organization that developed and trained the model. For example, OpenAI is the creator of GPT-4 and Meta is the creator of Llama 3.
- **Endpoint:** A hosted instance of a model that can be accessed via an API. A single model may have multiple endpoints across different providers.
- **System:** A dedicated compute environment for running AI models, typically provisioned infrastructure such as a VM, which can be benchmarked for performance under load.
- **Provider:** A company that hosts and provides access to one or more model endpoints or systems. Examples include OpenAI, AWS Bedrock, Together.ai and more. Companies are often both Model Creators and Providers.
- **Serverless:** Cloud service provided on an as-used basis, in relation to LLM inference APIs generally means priced per token of input and output. Serverless cloud products do still run on servers!
- **Open Weights:** A model whose weights have been released publicly by the model's creator. We refer to 'open weights' or just 'open' models rather than 'open-source' as many open LLMs have been released with licenses that do not meet the full definition of open-source software.
- **Token:** Modern LLMs are built around tokens - numerical representations of words and characters. LLMs take tokens as input and generate tokens as output. Input text is translated into tokens by a tokenizer. Different LLMs use different tokenizers.
- **Native Tokens:** Tokens as counted by an LLM's own tokenizer, usually as reported by the provider's API. Prices generally refer to native tokens, and our performance metrics, such as output speed, count them.
- **Price (Input/Output):** The price charged by a provider per input token sent to the model and per output token received from the model. Prices shown are the current prices listed by providers.
- **Price (Blended):** To enable easier comparison, we calculate a blended price assuming a 7:2:1 ratio of cache hit, input, and output tokens.

$$\frac{7 \times \text{Cache Hit Price} + 2 \times \text{Input Price} + \text{Output Price}}{10}$$

- **Cost per Task:** The weighted-average cost (USD) to complete one Artificial Analysis Intelligence Index task. This is calculated by multiplying input, cached, and output token prices by tokens consumed across the workload, weighted by the same per-benchmark weights the Intelligence Index uses, then dividing by task count. Price is either the provider's first-party API rate or the median across all providers. Caching

costs differ by provider. Because cost reflects actual token usage, models that produce longer answers or more reasoning tokens will have a higher cost per task, even at identical per-token prices.

$$\text{Cost per Task} = \sum_i w_i \cdot \frac{\text{Input Tokens}_i \cdot \text{Input Price} + \text{Cached Tokens}_i \cdot \text{Cache Price} + \text{Output Tok}}{\text{Tasks}_i}$$

where i = each benchmark in the Artificial Analysis Intelligence Index and w = that benchmark's weight in the Index

- **Time per Task:** The estimated weighted-average time to complete one Artificial Analysis Intelligence Index task. For each benchmark, we divide the answer and reasoning tokens per task by the model's output speed, then weight by the same per-benchmark weights the Intelligence Index uses. Output speed comes from our API performance benchmarks: output tokens per second over each request's full duration, averaged over the workloads we measured in the past 5 days, on the model's representative endpoint, usually the model creator's own API, which we also use for its price. When a model has no representative endpoint, or we have no measurements of it, we take the median across the model's measured endpoints. Where we show time per task for an endpoint, such as on provider pages, we use that endpoint's own output speed. We derive Time per Task from tokens and speed rather than timing evaluation runs, so it reflects representative API speeds.

$$\text{Time per Task} = \sum_i w_i \cdot \frac{\text{Answer Tokens}_i + \text{Reasoning Tokens}_i}{\text{Tasks}_i \cdot \text{Output Speed}}$$

where i = each benchmark in the Artificial Analysis Intelligence Index, w = that benchmark's weight in the Index, and Output Speed = the model's output speed in tokens per second

- **Time to First Answer Token:** The time in seconds between sending a request to the service or system and receiving the first answer token of the response. For reasoning models, this includes the time spent reasoning, so unlike time to first token, it measures the wait for an answer the same way for reasoning and non-reasoning models. For multi-turn workloads, we average it over a session's turns.

$$\text{Time to First Answer Token} = \text{Time of First Answer Token Arrival} - \text{Time Request Sent}$$

- **Time to First Token:** The time in seconds between sending a request to the service or system and receiving the first token of the response. For reasoning models which return reasoning tokens, this will be the first reasoning token. For multi-turn workloads, we average it over a session's turns.

$$\text{Time to First Token} = \text{Time of First Token Arrival} - \text{Time Request Sent}$$

- **Output Speed (output tokens per second):** The number of output tokens, including reasoning tokens, received per second over a request's full duration, from sending the request to receiving the final token, so it includes time to first token. For a multi-turn session, we divide its output tokens by the time the endpoint took to answer its requests, which excludes the time tools take to run between them.

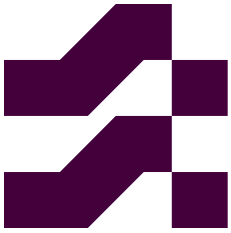
$$\text{Output Speed} = \frac{\text{Answer Tokens} + \text{Reasoning Tokens}}{\text{Time of Final Token Chunk Received} - \text{Time Request Sent}}$$

- **End-to-End Time:** The time in seconds between sending a request and receiving the final token, measured on each request. For a multi-turn session, it is the total over the session's requests, which excludes the time tools take to run between them.

$$\text{End-to-End Time} = \text{Time of Final Token Chunk Received} - \text{Time Request Sent}$$

- **Endpoint Error Rate:** The share of an endpoint's requests in the past 5 days that failed on the endpoint's side. A multi-turn session counts as one request, which fails if any of its requests fails. We exclude failures we don't attribute to the endpoint, such as an invalid API key, exhausted credits or our account's quota, a request rejected as invalid or too long for the context window, a response our max_tokens limit cuts off before it answers, a safety block or a sandbox error. Errors on the endpoint's side count, including its rate limits, timeouts and requests still running at the benchmark deadline. Across workloads, we pool requests before dividing.
- **Cache Hit Rate:** The share of cacheable input tokens served from the provider's prompt cache, as reported by the provider's API during multi-turn agentic sessions. We report a representative cache hit rate for each endpoint based on its recent measurements.

$$\text{Cache Hit Rate} = \frac{\text{Cached Input Tokens}}{\text{Cacheable Input Tokens}}$$



Artificial Analysis

Get notified about new articles

What's your email?

Explore

LLM Leaderboard
Image Arena
Video Arena
AI Agents
Evaluations

Products

Optima
MicroEvals
Model Recommender
Data Playground
Image Lab

Company

About
Methodology
Contact
Articles
Careers

© 2026 Artificial Analysis Terms of Use Data Platform Terms Privacy Policy

English